Forwarding reference to specific type/template

Document #:	P2481R1
Date:	2022-07-15
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

1 Revision History
2 Introduction
- 2.1 `std::tuple` converting constructor
- 2.2 `std::get` for `std::tuple`
- 2.3 `transform` for `std::optional`
- 2.4 `view_interface` members
- 2.5 The Goal

3 Not a proposal
- 3.1 `T auto&&`
- 3.2 `T&&&`
- 3.3 `const(bool)`
- 3.4 `qualifiers Q`
- 3.5 Circle's approach
- 3.6 Something else

4 References

§ 1 Revision History

Since [P2481R0], added Circle's approach and a choice implementor quote.

§ 2 Introduction

There are many situations where the goal of a function template is deduce an arbitrary type - an arbitrary range, an arbitrary value, an arbitrary predicate, and so forth. But sometimes, we need something more specific. While we still want to allow for deducing `const` vs mutable and lvalue vs rvalue, we know either what concrete type or concrete class template we need - and simply want to deduce *just* that. With the adoption of [P0847R7], the incidence of this will only go up.

It may help if I provide a few examples.

§ 2.1 `std::tuple` converting constructor

`std::tuple<T...>` is constructible from `std::tuple<U...>` *cv ref* when `T...` and `U...` are the same size and each `T` is constructible from `U` *cv ref* (plus another constraint to avoid clashing with other constructors that I'm just going to ignore for the purposes of this paper). The way this would be written today is:

```
template <typename... Ts>
struct tuple {
    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us&> && ...)
    tuple(tuple<Us...>&);

    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us const&> && ...)
    tuple(tuple<Us...> const&);

    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us&&> && ...)
    tuple(tuple<Us...>&&);

    template <typename... Us>
        requires sizeof...(Ts) == sizeof...(Us)
              && (is_constructible_v<Ts, Us const&&> && ...)
    tuple(tuple<Us...> const&&);
};
```

This is pretty tedious to say the least. But it also has a subtle problem: these constructors are all *overloads* - which means that if the one you'd think would be called is valid, it is still possible for a different one to be invoked. For instance:

```
void f(tuple<int&>);
auto g() -> tuple<int&&>;

void h() {
    f(g()); // ok!
}
```

Here, we're trying to construct a `tuple<int&>` from an *rvalue* `tuple<int&&>`. The desired behavior is that the `tuple<Us...>&&` is considered and then rejected (because `int&` is not constructible from `Us&&` - `int&&`). That part indeed happens. But the `tuple<Us...> const&` constructor still exists and that one ends up being fine (because `int&` is constructible from `Us const&` - which is `int&` here). That's surprising and undesirable.

But in order to avoid this, we'd need to only have a single constructor template. What we *want* is to have *some kind* of `tuple<Us...>` and just deduce the *cv ref* part. But our only choice today is either the code above (tedious, yet mostly functional, despite this problem) or to go full template:

```
template <typename... Ts>
struct tuple {
    template <typename Other>
        requires is_specialization_of<remove_cvref_t<Other>, tuple>
```

```
                && /* ???? */
    tuple(Other&& rhs);
};
```

How do we write the rest of the constraint? We don't really have a good way of doing so. Besides, for types which inherit from `std::tuple`, this is now actually wrong - that derived type will not be a specialization of tuple, but rather instead inherit from one. We have to do extra work to get that part right, as we currently do in the `std::visit` specification - see 22.6.7 [variant.visit]/1.

2.2 `std::get` for `std::tuple`

We run into the same thing for non-member functions, where we want to have `std::get<I>` be invocable on every kind of tuple. Which today likewise has to be written:

```
template <size_t I, typename... Ts>
auto get(tuple<Ts...>&) -> tuple_element_t<I, tuple<Ts...>>&;

template <size_t I, typename... Ts>
auto get(tuple<Ts...> const&) -> tuple_element_t<I, tuple<Ts...>> const&;

template <size_t I, typename... Ts>
auto get(tuple<Ts...>&&) -> tuple_element_t<I, tuple<Ts...>>&&;

template <size_t I, typename... Ts>
auto get(tuple<Ts...> const&&) -> tuple_element_t<I, tuple<Ts...>> const&&;
```

This one we could try to rewrite as a single function template, but in order to do that, we need to first coerce the type down to some kind of specialization of `tuple` - which ends up requiring a lot of the same work anyway.

2.3 `transform` for `std::optional`

The previous two examples want to deduce to some specialization of a class template, this example wants to deduce to a specific type. One of the motivation examples from "deducing `this`" was to remove the quadruplication necessary when writing a set of overloads that want to preserve `const`-ness and value category. The adoption of [P0798R8] gives us several such examples.

In C++20, we'd write it as:

```
template <typename T>
struct optional {
    template <typename F> constexpr auto transform(F&&) &;
    template <typename F> constexpr auto transform(F&&) const&;
    template <typename F> constexpr auto transform(F&&) &&;
    template <typename F> constexpr auto transform(F&&) const&&;
};
```

complete with quadruplicating the body. But with deducing this, we might consider writing it as:

```
template <typename T>
struct optional {
```

```
    template <typename Self, typename F> constexpr auto transform(this
        Self&&, F&&);
};
```

But this deduces too much! We don't want to deduce derived types (which in addition to unnecessary extra template instantiations, can also run into shadowing issues). We *just* want to know what the `const`-ness and value category of the `optional` are. But the only solution at the moment is to C-style cast your way back down to your own type. And that's not a particular popular solution, even amongst standard library implementors:

> FWIW, we're no longer using explicit object member functions for `std::expected`; STL couldn't stomach the necessary C-style cast without which the feature is not fit for purpose.

§ 2.4 `view_interface` members

Similar to the above, but even more specific, are the members for `view_interface` (see 26.5.3 [view.interface]). Currently, we have a bunch of pairs of member functions:

```
template <typename D>
class view_interface {
    constexpr D& derived() noexcept { return static_cast<D&>(*this); }
    constexpr D const& derived() const noexcept { return static_cast<D
        const&>(*this); }

public:
    constexpr bool empty() requires forward_range<D> {
        return ranges::begin(derived()) == ranges::end(derived());
    }
    constexpr bool empty() const requires forward_range<D const> {
        return ranges::begin(derived()) == ranges::end(derived());
    }
};
```

With deducing this, we could write this as a single function template - deducing the self parameter such that it ends up being the derived type. But that's again deducing way too much, when all we want to do is know if we're a `D&` or a `D const&`. But we can't deduce just `const`-ness.

§ 2.5 The Goal

To be more concrete, the goal here is to be able to specify a particular type or a particular class template such that template deduction will *just* deduce the `const`-ness and value category, while also (where relevant) performing a derived-to-base conversion. That is, I want to be able to implement a single function template for `optional<T>::transform` such that if I invoke it with an rvalue of type `D` that inherits publicly and unambiguously from `optional<int>`, the function template will be instantiated with a first parameter of `optional<int>&&` (not `D&&`).

§ 3 Not a proposal

If you don't find the above examples and the need for more concrete deduction motivating, then I'm sorry for wasting your time.

If you *do* find the above examples and the need for more concrete deduction motivating, then I'm sorry that I don't actually have a solution for you. What I have instead are several example syntaxes that I've thought about over the years that are all varying degrees of mediocre. My hope with this paper is that other, more creative, people are equally interesting in coming up with a solution to this problem and can come up with a better syntax for it.

Here are those syntax options. I will, for each option, demonstrate how to implement the `tuple` converting constructor, `optional::transform`, and `view_interface::empty`. I will use the following tools:

```
#define FWD(e) static_cast<decltype(e)&&>(e)

template <bool RV, typename T>
using apply_ref = std::conditional_t<RV, T&&, T&>;

template <bool C, typename T>
using apply_const = std::conditional_t<C, T const, T>;

template <bool C, bool RV, typename T>
using apply_const_ref = apply_ref<RV, apply_const<C, T>>;

template <typename T, typename U>
using copy_cvref_t = apply_const_ref<
  is_const_v<remove_reference_t<T>>,
  !is_lvalue_reference_v<T>,
  U>;
```

§ 3.1 `T auto&&`

The principle here is that in the same say that `range auto&&`is some kind of `range`, that `int auto&&` is some kind of `int`. It kind of makes sense, kind of doesn't. Depends on how you think about it.

tuple	`template <typename... Ts>`
	`struct tuple {`
	`  template <typename... Us>`
	`  tuple(tuple<Us...> auto&& rhs)`
	`  requires sizeof...(Us) == sizeof...(Ts)`
	`  && (constructible_from<`
	`  Ts,`
	`  copy_cvref_t<decltype(rhs), Us>`
	`  > && ...);`
	`};`

optional	`template <typename T>`
	`struct optional {`
	`  template <typename F>`
	`  auto transform(this optional auto&& self, F&& f) {`

	using U = remove_cv_t<invoke_result_t<F,
	decltype(FWD(self).value())>;
	
	if (self) {
	return optional<U>(invoke(FWD(f), FWD(self).value()));
	} else {
	return optional<U>();
	}
	}
	};

view_interface	template <typename D>
	struct view_interface {
	constexpr bool empty(this D auto& self)
	requires forward_range<decltype(self)>
	{
	return ranges::begin(self) == ranges::end(self);
	}
	};

The advantage of this syntax is that it's concise and lets you do what you need to do.

The disadvantage of this syntax is that the only way you can get the type is by writing `decltype(param)` - and the only way to can pass through the `const`-ness and qualifiers is by grabbing them off of `decltype(param)`. That's fine if the type itself is all that is necessary (as it the case for `view_interface`) but not so much when you actually need to apply them (as is the case for `tuple`). This also means that the only place you can put the `requires` clause is after the parameters. Another disadvantage is that the derived-to-base conversion aspect of this makes it inconsistent with what `Concept auto` actually means - which is not actually doing any conversion.

§ 3.2 `T&&&`

Rather than writing `tuple<U...> auto&& rhs` we can instead introduce a new kind of reference and spell it `tuple<U...>&&& rhs`. This syntactically looks nearly the same as the `T auto&&` version, so I'm not going to copy it.

If we went this route, we would naturally have to also allow:

```
template <typename T> void f(T&&);  // regular forwarding reference
template <typename T> void g(T&&&); // also regular forwarding reference
```

The advantage here is that it's less arguably broken than the previous version, since it's more reasonable that the `tuple<U...>&&&` syntax would allow derived-to-base conversion.

The disadvantages are all the other disadvantages of the previous version, plus also a whole new reference token? Swell.

§ 3.3 `const(bool)`

We have `noexcept(true)` and `explicit(true)`. What about `const(true)`?

On some level, this seems to make perfect sense. At least for `const` - since we want to deduce either `T` or `T const`, and so `const` is either absent or present. But what about value category? How do you represent `T&` vs `T&&`? Surely, we wouldn't do `T &(LV) &&(RV)` for deducing two different `bool`s - these two cases are mutually exclusive. Keeping one of the `&`s around, as in `T& &(RV)` (with a mandatory space) also seems pretty bad. So for the purposes of this section, let's try `T && (RV)` (where `RV` is `true` for rvalues and `false` for lvalues, but still a forwarding reference).

tuple

```
template <typename... Ts>
struct tuple {
  template <typename... Us, bool C, bool RV>
    requires sizeof...(Us) == sizeof...(Ts)
          && (constructible_from<
                Ts,
                apply_const_ref<C, RV, Us>
              > && ...)
  tuple(tuple<Us...> const(C) &&(RV) rhs);
};
```

optional

```
template <typename T>
struct optional {
    template <typename F, bool C, bool RV>
    auto transform(this optional const(C) &&(RV) self, F&& f) {
        using U = remove_cv_t<invoke_result_t<F,
            // apply_const_ref<C, RV, T>
            decltype(FWD(self).value())>;

        if (self) {
          return optional<U>(invoke(FWD(f), FWD(self).value()));
        } else {
          return optional<U>();
        }
    }
};
```

view_interface

```
template <typename D>
struct view_interface {
    template <bool C>
        requires forward_range<apply_const<C, D>>
    constexpr bool empty(this D bool(C)& self)
    {
        return ranges::begin(self) == ranges::end(self);
    }
};
```

This syntax is… pretty weird. Very weird.

The advantages are that it's clearer that we're only deducing `const`-ness and ref qualifiers. It also allows you to put `requires` clauses after the *template-head* rather than much later. When only deducing `const`, it's arguably pretty clear what's going on.

The disadvantages are the obvious weirdness of the syntax, *especially* for figuring out the value category, and the mandatory metaprogramming around applying those boolean values that we deduce through the types. `apply_const` and `apply_const_ref` (as I'm arbitrarily calling them here, the former appears as an exposition-only trait in Ranges under the name *`maybe-const`*) will be *everywhere*, and those aren't exactly obvious to understand either. It may be tempting to allow writing `int const(false) &&(true)` as a type directly to facilitate writing such code (this would be `int&&`), but this seems facially terrible.

There's further issues that `int const(true)` isn't quite valid grammar today, but it's pretty close. `const(true)` looks like a cast, and it's not unreasonable that we may at some point consider `const(x)` as a language cast version of `std::as_const(x)`.

But there is one entirely unrelated benefit. Consider trying to write a function that accepts any function pointer:

```
template <typename R, typename... Args, bool B>
void accepts_function_ptr(R (*p)(Args...) noexcept(B));
```

That's all you need (if we ignore varargs, assume the adoption of [CWG2355] - which was accepted but not fully processed yet). And note here the deduction of `noexcept`-ness (and the usage of it on a function type) very closely resembles the kind of deduction of `const` and value category discussed in this section.

But today if we wanted to deduce a pointer to *member* function, we have to write a whole lot more - precisely because we can't deduce all the other stuff at the end of the type:

```
// this only accepts non-const pointers to member functions that have no
        ref-qualifier
template <typename R, typename C, typename... Args, bool B>
void accepts_member_function_ptr(R (C::*p)(Args...) noexcept(B));

// so we also need this one
template <typename R, typename C, typename... Args, bool B>
void accepts_member_function_ptr(R (C::*p)(Args...) const noexcept(B));

// ... and this one
template <typename R, typename C, typename... Args, bool B>
void accepts_member_function_ptr(R (C::*p)(Args...) & noexcept(B));

// ... and also this one
template <typename R, typename C, typename... Args, bool B>
void accepts_member_function_ptr(R (C::*p)(Args...) && noexcept(B));

// ... and then also these two
template <typename R, typename C, typename... Args, bool B>
void accepts_member_function_ptr(R (C::*p)(Args...) const& noexcept(B));

template <typename R, typename C, typename... Args, bool B>
void accepts_member_function_ptr(R (C::*p)(Args...) const&& noexcept(B));
```

The direction where we can deduce `const` in the same way that we can deduce `noexcept` provides a much better solution for this. Although here, unlike the examples presented earlier, we're not simply selecting between `&` and `&&`. Here, we have three options. Does that mean a different design then? And what would it look like to handle both cases? It's a bit unclear.

§ 3.4 `qualifiers Q`

This approach is quite different and involves introducing a new kind of template parameter, which I'm calling `qualifiers`, which will deduce an *alias template*. It may be easier to look at the examples:

tuple

```
template <typename... Ts>
struct tuple {
  template <typename... Us, qualifiers Q>
    requires sizeof...(Us) == sizeof...(Ts)
          && (constructible_from<Ts, Q<Us>> && ...)
  tuple(Q<tuple<Us...>> rhs);
};
```

optional

```
template <typename T>
struct optional {
    template <typename F, qualifiers Q>
    auto transform(this Q<optional> self, F&& f) {
        using U = remove_cv_t<invoke_result_t<F, Q<T>>>;

        if (self) {
          return optional<U>(invoke(FWD(f), FWD(self).value()));
        } else {
          return optional<U>();
        }
    }
};
```

view_interface

```
template <typename D>
struct view_interface {
    template <qualifiers Q>
        requires forward_range<Q<D>>
    constexpr bool empty(this Q<D>& self)
    {
        return ranges::begin(self) == ranges::end(self);
    }
};
```

The idea here is that a parameter of the form `Q<T> x` will deduce `T` and `Q` separately, but `Q` will be deduced as one of the following four alias templates:

- `template <typename T> using Q = T&;`
- `template <typename T> using Q = T const&;`
- `template <typename T> using Q = T&&;`

— `template <typename T> using Q = T const&&;`

Whereas a parameter of the form `Q<T>& x` or `Q<T>&&` will deduce `Q` either as:

— `template <typename T> using Q = T;`

— `template <typename T> using Q = T const;`

The significant advantage here is that applying the `const` and reference qualifiers that we just deduced is trivial, since we already have exactly the tool we need to do that: `Q`. This makes all the implementations simpler. It also gives you a way to name the parameter other than `decltype(param)`, since there is a proper C++ spelling for the parameter itself in all cases.

The disadvantage is that this is *quite* novel for C++, and extremely weird. Even more dramatically weird than the other two solutions. And that's even with using the nice name of `qualifiers`, which is probably untenable (although `cvrefquals` or `refqual` might be available?). Also `Q<T> x` does not look like a forwarding reference, but since `Q<T>& x` is the only meaningful way to deduce just `const` - this suggests that `Q<T>&& x` *also* needs to deduce just `const` (even though why would anyone write this), which leaves `Q<T> x` alone.

There's also the question of how we could provide an explicit template argument for `Q`. Perhaps that's spelled `qualifiers::rvalue` (to add `&&`) or `qualifiers::const_lvalue` (to add `const&`) and the like? There'd need to be some language magic way of spelling such a thing - since we probably wouldn't want to just allow an arbitrary alias template. `std::add_pointer_t`, for instance, would suddenly introduce a non-deduced context, and wouldn't make any sense anyway.

§ 3.5 Circle's approach

Sean Baxter implemented the following approach in Circle:

```
template<typename T, typename... Args>
void f(T&& y : std::tuple<Args...>);
```

In this syntax, we effectively are deducing some kind of `std::tuple<Args...>`. `T` here is always a (possibly reference to) (possibly const) `std::tuple`. This behaves the same way as the previous section's:

```
template<qualifiers Q, typename... Args>
void f(Q<std::tuple<Args...>> y);
```

with the only difference being what additional information you have available to you: whether that's the type of `y` (`T`, or really `T&&`) or an alias providing the appropriate qualifiers (`Q`). As such, their uses are broadly similar:

Qualifiers	Circle

`template <typename... Ts>`
`struct tuple {`
`  template <typename... Us,`
`  qualifiers Q>`
	`template <typename... Ts>`
	`struct tuple {`
	`  template <typename... Us, typename`
	`  Rhs>`

Qualifiers	Circle
`    requires sizeof...(Us) == sizeof...(Ts)  && (constructible_from<Ts, Q<Us>> && ...)  tuple(Q<tuple<Us...>> rhs); };`	`    requires sizeof...(Us) == sizeof...(Ts)  && (constructible_from<Ts, copy_cvref_t<Rhs, Us>> && ...)  tuple(Rhs&& rhs : tuple<Us...>); };`
`template <typename D> struct view_interface {  template <qualifiers Q>  requires forward_range<Q<D>>  constexpr bool empty(this Q<D>& self)  {  return ranges::begin(self) == ranges::end(self);  } };`	`template <typename D> struct view_interface {  template <class Self>  requires forward_range<Self>  constexpr bool empty(this Self& self : D)  {  return ranges::begin(self) == ranges::end(self);  } };`

The Circle approach requires a bit more work to propagate the qualifiers as compared to the qualifiers approach (which exists to propagate qualifiers), but gives you an actual name for the actual type you end up with rather than having to re-spell it manually.

As Circle's approach is conceptually similar to the qualifiers approach, it is weird for some of the same ways. The declaration `T&& t : tuple<U...>` may look like it's deducing `t` as a regular forwarding reference, but it's not - it additionally (potentially) undergoes a derived-to-base conversion at the call site. We now have two types attached to a given variable. For those coming from languages which put the type of a variable after a colon, it would look like the type of `t` is `tuple<U...>` - and that's kind of correct at least (the type of `remove_cvref_t<decltype(t)>` is `tuple<U...>`). It seems unlikely that we would ever move C++ into a direction with better declaration syntax, but if we did this would cut off that approach.

3.6 Something else

If there's a different approach someone has, I'd love to hear it. But this is what I've got so far.

4 References

[CWG2355] John Spicer. 2017-09-06. Deducing noexcept-specifiers. https://wg21.link/cwg2355

[P0798R8] Sy Brand. 2021. Monadic operations for std::optional. https://wiki.edg.com/pub/Wg21virtual2021-10/StrawPolls/p0798r8.html

[P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this. https://wg21.link/p0847r7

[P2481R0] Barry Revzin. 2021-10-15. Forwarding reference to specific type/template.
https://wg21.link/p2481r0